Products Experience — Design Specification (rev 3)

Two-tier showcase, feature blocks (image + explanation), related blogs, flexible admin

Date:	2026-07-04 (rev 3: feature showcase; rev 2 hardened after review)
Status:	Approved (design phase)
Owner:	Fatima Abdul Raheem (CEO)
Depth decision:	Content + presentation control

Contents

1 Overview

Build the **products experience**: a public product showcase driven end-to-end by a flexible admin. The showcase is **two-tier by design** — an approachable **Quick View** (a box that opens on the same page, plain language, for everyone) and a **full technical page** (for technical readers). Each product can also reference **related blog posts**, so readers flow from product → deep-dive articles.

The admin controls both **what** shows (content, media, related posts) and **how** it's arranged (order, pinning, categories, publish state) — with drafts and preview. The editor is organized around the two public tiers so the generic tier is quick and foolproof to fill.

Ground truth: the original site-spec schema is already live on project `axbsghyqhhdaiylcksbv` (migrations `0001+0002`: `profiles`, `products`, `blog_posts`, `orders`, storage policies, admin auth, seeded products). This spec therefore defines a **data-preserving migration** of that schema, never a fresh install.

1.1 Why this shape (from the 2026-07-03 admin-flexibility research)

Reference gap	Our answer
No ordering/pinning	`sort_order` + drag-to-reorder; featured pinning
Everything live on save	Draft → Published workflow + preview
Hardcoded category dropdown	Admin-managed categories table
No detail pages / SEO	`/products/[slug]` pages with per-product SEO
No live preview	Preview renders the real public pages from draft data
One-size-fits-all cards	Two-tier: Quick View for everyone, technical page for devs

1.2 Goals

- Two-tier presentation: plain-language Quick View first; technical depth one click away.
- Public showcase that sells: filterable grid + rich detail pages linking to live apps.
- Products cross-link to related blog posts (and posts link back when the blog ships).
- Every product can showcase its features visually — picture + explanation blocks, added and ordered from the admin.
- Admin reshapes the grid without a deploy, fills the generic tier fast, and can never leave a half-empty public page (validation on publish **and** on edits to published products).
- Safe publishing: draft/preview/publish; nothing accidental.
- Schema evolution that keeps today's six live seed products rendering throughout.

1.3 Non-goals (deferred)

- Blog **admin** & orders inbox (separate specs; `blog_posts` already exists and the product↔post links ship here).
- Payments, client accounts.
- Full site-builder — revisit after products ship.

2 Public UX

2.1 `/products` — showcase grid

- Card: cover image, name, **tagline**, category pill, lifecycle badge (*Beta*/*Soon* when applicable), Featured badge. Tech chips live in the technical tier (intentionally replaces today's tag chips on cards).
- Filter pills from the categories table (+ "All"); empty categories hidden.
- Order: `featured DESC, sort_order ASC, created_at DESC` (deterministic).
- **Cards are real links** (`<a href="/products/[slug]">`) whose click is intercepted to open the Quick View — crawlers and no-JS users get the detail page (SEO-safe); JS users get the overlay.
- Server Component; `revalidatePath` on admin mutations keeps it fresh.

2.2 Quick View — tier 1, for everyone

- Dialog over the grid: cover, name, category, **tagline**, **highlights** (3–6 plain-language bullets), CTA row (rules below) + **Full technical details** → (real navigation to `/products/[slug]`), plus up to two related-post chips.
- **URL mechanics:** opening writes `?p=<slug>` via `history.pushState` (no RSC refetch); open state hydrates client-side from `useSearchParams`, so the page stays cacheable. Back/Esc/backdrop close. Direct visits to `/products?p=<slug>` open the grid with the box open. Unknown, draft, or malformed `?p=` is **silently ignored**. `/products` keeps `rel="canonical"` to itself regardless of `?p=`; every published product appears in the sitemap as `/products/[slug]`.
- No extra fetch: quick-view fields ride the grid query. Focus-trapped, `role="dialog"`, labeled by product name.
- Content rule: **zero jargon** — admin fields are labeled accordingly.

2.3 `/products/[slug]` — tier 2, the technical page

- **Overview strip first** (cover, tagline, highlights) so non-technical visitors who land directly aren't lost.
- **Feature showcase** — each product feature as a picture + explanation block (image, title, short text), rendered in admin-set order as alternating image/text rows; hidden when a product has no features.
- Technical body: long description (Markdown), **Technical details** (Markdown), tech chips, gallery grid, CTAs (rules below).
- **Related posts**: cards for linked *published* posts → `/blog/[slug]`; hidden when none. (Blog pages get a "Built product" backlink when the blog admin ships.)
- **"Request something like this"** CTA → `/order?ref=<slug>`.
- SEO: `generateMetadata` from `seo_description` (fallback: tagline+summary) + OG image (fallback: cover). Drafts 404 publicly.

2.4 CTA rules (both tiers)

- `live_url` set **and** `lifecycle != 'soon'` → primary **Open live app**.
- `lifecycle = 'soon'` or `live_url` empty → disabled **Coming soon** state (never a dead link). `live_url` stays optional at publish time by design.
- `repo_url` set → secondary **View repo** (technical page).

3 Admin UX (`/admin/products`)

- **List view:** cover thumb, name, category, status badge, lifecycle, Featured star, **drag-to-reorder** (persists `sort_order`); filter by status/category; **Duplicate** action. The list groups featured items first to mirror public order; **reordering is disabled while a filter is active** (positions are global).
- **Editor — organized by audience tier** (tabs mirroring the public surfaces):
  1. **Quick View tab** *(the box everyone sees first)*: name, slug (auto, editable), tagline *(card + quick view headline)*, summary *(quick view intro)*, **highlights list editor** (add/remove/drag-reorder, "plain language" helper text), category select, cover upload, live URL, lifecycle select (live/beta/soon), featured toggle.
  2. **Features tab** *(the detail page's showcase)*: repeating feature rows — image upload, title, explanation text; add/remove/drag-reorder. Optional: products may publish with zero features.
  3. **Technical tab** *(the detail page)*: long description (Markdown), technical details (Markdown), tech list, gallery uploads, repo URL, year.
  4. **SEO & Links tab**: SEO description *(search results)*, OG image *(social shares)*, **Related blog posts picker** (search, multi-select, drag-order; drafts selectable but flagged).
- **Publish validation** — Quick View tier must be complete (name, tagline, summary, $\geq$3 highlights, cover, category) for a product to be **or remain** published: Publish enables only when valid; **saving edits to a published product re-runs the validation** (invalid save blocked, with *Unpublish & save as draft* offered); drafts save anytime.
- **Preview** (admin-only): renders the *actual* public components for both tiers from the **last saved draft** (save-first; `/admin/products/[id]/preview` fetches with the admin client). Public tier components are **pure/prop-driven**; routes fetch and pass data in.
- **Duplicate** semantics: new **draft**; slug `-copy`/`-copy-2` with uniqueness retry; `featured=false`; `sort_order` = end; links copied; **image URLs copied by reference** (shared Storage objects — future image-cleanup work must check referrers; flagged).
- **Categories manager:** add/rename/delete/reorder; delete blocked while referenced (reassign first).
- Uploads $\rightarrow$ `product-images` bucket (policies already live from 0001).

4 Data model (Supabase)

4.1 Migration path — `0003` transforms the live schema (data-preserving)

The DB already holds `products` with `published bool`, `status text default 'Live'` (Live/Beta/Soon), `category text`, `tags text[]`, `year text`, and six seeded rows. Migration `0003` (single transaction):

1. `product_categories` created; distinct existing `category` strings inserted; `category_id` FK added + backfilled by name; old text column dropped.
2. Existing `status` **renamed to** `lifecycle`, values lowercased, `check in ('live','beta','soon')`.
3. New `status check in ('draft','published')` derived from `published` (`true` $\rightarrow$ `'published'`); `published_at` backfilled; `published bool` dropped. (CHECK added **after** backfill — adding it first would fail against existing rows.)
4. `tags` **renamed to** `tech`.

5. New columns: `tagline`, `highlights text[]`, `technical_details`, `gallery text[]`, `sort_order int` (backfilled 10, 20, 30… by `created_at` so the grid is stable and insertable), `seo_description`, `og_image`.
6. `product_blog_links` junction and `product_features` table created.
7. Policy + trigger fixes from §5.

Seed data survives every step; `/products` renders identically before and after.

4.2 Final shapes

`product_categories` — `id uuid PK · name · slug unique · sort_order · created_at`

`products` — `id uuid PK · slug unique · name · tagline` *(quick-view headline)* · `summary` *(quick-view intro)* · `highlights text[]` *(quick-view bullets)* · `description` (Markdown) · `technical_details` (Markdown) · `cover_image · gallery text[] · category_id FK` (nullable) · `tech text[] · lifecycle check in ('live','beta','soon') · year · live_url · repo_url · featured bool · sort_order int · status check in ('draft','published') · seo_description · og_image · published_at ·` timestamps

`blog_posts` — already live (0001/0002); unchanged here.

`product_blog_links` — `product_id FK · blog_post_id FK · sort_order ·` PK `(product_id, blog_post_id) ·` FKs `ON DELETE CASCADE`

`product_features` (new) — `id uuid PK · product_id FK ON DELETE CASCADE · title · description` *(the explanation shown beside the picture)* · `image · sort_order · created_at`

`profiles` — already live; role-protection hardened (§5).

4.3 Indexes

Existing kept; added: `products(category_id)`, `products(sort_order)`, `products(lifecycle)`, `product_categories(slug)`, `product_blog_links(product_id)`, `product_blog_links(blog_post_id)`, `product_features(product_id, sort_order)`.

4.4 RLS (default-deny, every table)

- `products`: public SELECT where `status='published'` (policy updated from the old bool); admins full.
- `product_categories`: public SELECT; admins full.
- `blog_posts`: unchanged.
- `product_blog_links`: public SELECT **only when both sides are published** (`EXISTS` on published product **and** published post) — anon users cannot count posts attached to unreleased products or see draft-post UUIDs; admins full.
- `product_features`: public SELECT only when the parent product is published (`EXISTS` on published parent); admins full.
- `profiles`: self-read; admins read all; **role changes owner-gated by trigger** (§5).
- Storage `product-images`: unchanged from 0001.

5 Auth & guarding

- Supabase Auth email/password (already live); `/admin/login` + guarded panel layout reused.

- **Privilege-escalation fix (0003):** the 0001 self-update policy lets any signed-in user set their own `role`. 0003 adds a `BEFORE UPDATE` trigger on `profiles` rejecting `role` changes unless the caller is an `owner` (a trigger, not RLS, because RLS is row-granular and cannot see which column changed).
- `service_role` never leaves the server; Server Actions re-verify the session.

6 Performance

- Server Components throughout; client leaves only where interactive (Quick View dialog, drag list, editor, uploads).
- Quick View: no extra request; `pushState`-only open/close (no RSC refetch).
- Public pages cached; mutations call `revalidatePath` — publish is instant.
- `next/image` everywhere.
- Drag-and-drop: `@dnd-kit` (keyboard + touch accessible, which native HTML5 DnD lacks). The one new dependency.

7 Testing

- Unit: slug generation + duplicate-suffix retry, ordering tiebreaks, status transitions, publish-validation (incl. edits to published), category-delete guard.
- Component: card (link + intercepted click), grid filtering, Quick View (open/close/Back, `?p=` deep link, invalid `?p=` ignored), CTA rules (live/soon/no-URL), editor tabs + labels, related-posts hidden when empty, feature showcase renders in admin order / hides when empty.
- RLS impersonation: anon sees only published (incl. junction both-sides rule); anon writes fail; non-owner cannot change `profiles.role` (trigger test).
- Migration: 0003 dry-run against a copy of live data; `/products` renders identically pre/post.
- Manual: draft → preview (both tiers) → publish; reorder reflects publicly; unpublish 404s the detail page.

8 Build order (three PRs)

1. `feat/products-schema` — migration 0003 (transform live schema: categories, lifecycle/status/tech renames, new columns, junction, **features table**, RLS updates, role trigger, indexes, `sort_order` backfill); typed query/view-model updates so the current UX keeps rendering unchanged.
2. `feat/products-public` — two-tier UX: grid + Quick View (`?p=` pushState deep links) + technical page with **feature showcase**, related posts, CTA rules, SEO + sitemap, order-form CTA, empty states; public tier components prop-driven (preview-ready).
3. `feat/products-admin` — tiered CRUD editor (Quick View / **Features** / Technical / SEO & Links tabs) with publish validation (incl. published-edit revalidation), draft/preview/publish, drag reorder (`@dnd-kit`), categories manager, related-posts picker, duplicate action, image uploads, revalidation.

Each PR ships the full green bar (typecheck, lint, tests, build) + screenshots before merge.